Tutorial 2

FitchFork and Testing

Tutorial Structure

1. Recap from tutorial 1
2. Testing: Case study
3. Testing: Breakdown
4. Testing: Notes
5. Fitchfork

1. Recap

- Deadlines and Instructions
- Plagiarism
- Implementation:
  - Today’s program
  - Some things we need to know
- Testing our example code
- Submission

2. Case Study: The problem

The faculty is trying to develop a software tool that can help undergraduate students to calculate an estimate of their tuition fees.

The program prompts the student for the following inputs:

- Course enrolled
- Academic level

And prints out the final estimate figure of their tuition fees for the student.

2. Case Study: The requirements

The fees are calculated as follow:

- Registration
  - Each student takes a total of 2 Modules in the following categories
    - A single module by the university for computer literacy (Required)
    - The remaining module is course specific ie. course module
- Cost
  - The base cost for a **first year module** is R1000
  - Any module from the EBIT faculty cost ⅓ **more than the base cost**. (EBIT Faculty Increase)
  - Modules for each academic year level's base cost is ¼ more than the **preceding academic year** base cost

3. Break down for testing

Only consider the first requirement:

1. The base cost for a **first year module** is R1000

Valid Combinations:

Registered Modules	Cost
	Base
1. Computer Literacy	✓
2. Course Module	✓

- Test input to produce different results:
  - Course enrolled: all modules have the same cost
  - Academic level: academic level is irrelevant to the cost
  - Therefore all students will pay the same amount

3. Break down for testing

Consider the first two requirements:

1. The base cost for a **first year module** is R1000
2. Any module from the EBIT faculty cost ⅓ **more than the base cost**.

Registered Modules	Cost	
	Base	EBIT Faculty Increase
1. Computer Literacy	✓	X
2. Course Module	✓	✓

- Test input to produce different results:
  - Course Module enrolled: EBIT or Other Faculty
  - Academic level: academic level is irrelevant to the cost
  - Therefore: 2 minimum test case:
    - Computer Literacy + Non-Ebit Course Module ⟶ R1000 + R1000 = R2000
    - Computer Literacy + Ebit Course Module ⟶ R1000 + R1333 = R2333 (rounded down)

3. Break down for testing

Consider all requirements:

1. The base cost for a **first year module** is R1000
2. Any module from the EBIT faculty cost ⅓ **more than the base cost**.
3. Modules for each academic year level cost ¼ more than the **preceding academic year cost** (assume 4 years and both modules have to be on the same course year level)

Registered Modules	Cost	
	Base Cost	EBIT Faculty Increase
1. Computer Literacy	✓ + Year Increase	X
2. Course Module	✓ + Year Increase	✓ + Year Increase

Year Increase can take *4 possible values*: For Year 1 to 4

- Test input to produce different results:
  - Course Module enrolled: EBIT or Other Faculty
  - Academic level: Year 1-4 Increase
  - Therefore: 2 (EBIT or Other Faculty) x 4 (Year 1,2,3,4) = 8 minimum tests cases
  - (What if modules didn't have to be on the same course year level?)
    - 4 (Computer Literacy) * 4 (Non-EBIT Faculty Course Modules) + 4 (Computer Literacy) * 4(EBIT Faculty Course)

4. Some other notes on testing

- Test that your code handles the following according to specification:
  - Invalid Operations ie. How to handle errors:
    - Eg. division by 0
    - The spec will tell you what we want you to do in that case
  - Boundary Cases:
    - If you are developing a form that accepts ages 1 through 100, test what happens when someone enters '0', '1', '100', ‘-2’, and '101'.
  - Edge Cases
    - Test unexpected or least likely scenarios to ensure the system works under every possible condition.
    - Eg. You are writing a program to scan people’s IDs to see that they are old enough to buy alcohol. It should allow someone to buy alcohol at 00:00:00 on the day they turn 18. It shouldn’t allow someone to buy alcohol at 23:59:59 the day before they turn 18.
  - Zero and Negative Values
    - Make sure to always test with 0 and negative values.

5. FitchFork

- We use FF as our Tester.
- We set it up with a lot of different cases (the same way we worked out cases earlier) and then test that your code handles it as required by the specification.
- It is important that you test as many of these cases before you submit as you can

Live Demo of FitchFork:

- A Demo practical has been released on FF
- Please follow along while we go through doing the practical and submitting to FF
- This should help you complete Practical 1 if you haven’t yet